

Six RSA Attacks: Håstad, Common Modulus, Bleichenbacher, Shared Primes, Fermat, and Pollard's $p - 1$

Every attack below has the same shape: RSA is used with a small mistake in *how it is deployed or how its keys are generated*, and a tool from Lesson 1 — the Chinese Remainder Theorem, the Extended Euclidean Algorithm, RSA's multiplicative malleability, the plain gcd, the difference of two squares, or Fermat's little theorem — turns that mistake into a full break. The first four attacks never factor n at all. The last two do factor n , but only because a flawed key generator made n much easier to factor than a product of two independent random primes. None of them breaks correctly generated and correctly used RSA.

Attack	The mistake	Tool used	Fix
1. Håstad	Same message, small e , many recipients	CRT	Randomized padding (OAEP)
2. Common modulus	One n shared by several users	Extended Euclid	Never share n
3. Bleichenbacher	Server reveals “padding valid?”	Malleability	OAEP, uniform errors
4. Shared prime	Bad randomness reuses one prime p	gcd (Euclid)	Good entropy at key generation
5. Fermat	Primes p, q chosen too close together	Difference of squares	Pick p, q independently at random
6. Pollard $p - 1$	$p - 1$ has only small prime factors	Fermat's little theorem + gcd	Primes with a large prime factor in $p - 1$

1. Håstad's Broadcast Attack

Picture it

Think of $m^3 \bmod n$ as the reading on a clock with n hours. If m^3 is bigger than n , the hand goes around the dial and you only see where it ended up — that wraparound is what hides m . Now suppose the same m^3 is shown to you on *three different clocks*, of sizes n_1, n_2, n_3 . CRT lets you merge them into one giant clock of size $n_1 n_2 n_3$. If m^3 is smaller than that giant clock, the hand never wrapped, so the reading *is* m^3 , and a plain cube root gives m . Small e means too little wraparound to hide anything.

When this arises. One message must go to several people: a vendor sends the same license key or firmware secret to many customers, or a group announcement is sent encrypted to each member. Every recipient has their own independent key pair, but many use the same tiny exponent (historically $e = 3$, chosen because it makes encryption fast on weak hardware). The sender encrypts the identical m separately under each recipient's public key.

Setup. Exponent $e = 3$, the same message m sent to three recipients with pairwise-coprime moduli n_1, n_2, n_3 . The attacker sees

$$c_i = m^3 \bmod n_i, \quad i = 1, 2, 3.$$

Theorem 1. *If $m < \min(n_1, n_2, n_3)$, an eavesdropper who sees c_1, c_2, c_3 recovers m with no private key.*

Why it works.

1. CRT combines the three congruences $x \equiv c_i \pmod{n_i}$ into one unique answer C modulo $N = n_1 n_2 n_3$. This uses only public data.
2. The true value m^3 satisfies all three congruences too, so by uniqueness $C \equiv m^3 \pmod{N}$.
3. Because $m < n_i$ for each i , we have $m^3 < n_1 n_2 n_3 = N$. So C and m^3 are both in $[0, N)$ and congruent mod N , hence *equal as ordinary integers*.
4. Take an ordinary integer cube root of C . Done.

Worked example. Take $n_1 = 143 = 11 \cdot 13$, $n_2 = 323 = 17 \cdot 19$, $n_3 = 667 = 23 \cdot 29$, and secret message $m = 17$. The three ciphertexts are

$$c_1 = 17^3 \bmod 143 = 51, \quad c_2 = 17^3 \bmod 323 = 68, \quad c_3 = 17^3 \bmod 667 = 244.$$

Each number alone looks unrelated to 17. The attacker now runs CRT with $N = 143 \cdot 323 \cdot 667 = 30,808,063$. For each i , let $N_i = N/n_i$ and $y_i = N_i^{-1} \bmod n_i$:

n_i	c_i	N_i	y_i
143	51	215,441	112
323	68	95,381	286
667	244	46,189	221

$$C = 51 \cdot 215441 \cdot 112 + 68 \cdot 95381 \cdot 286 + 244 \cdot 46189 \cdot 221 \bmod N = 4913.$$

Since $17^3 = 4913$ is far less than $N \approx 3 \times 10^7$, there was no wraparound at all, and $\sqrt[3]{4913} = 17 = m$.

Takeaway and fix

Never send the same (or related) message to several recipients under a small e without randomized padding. With OAEP each ciphertext encrypts a *different* random-looking padded value, so the three equations no longer share one m^3 . (In general, with exponent e the attacker needs e recipients.)

2. Common Modulus Attack

Picture it

This is the classic water-jug puzzle. With a 17-litre jug and a 7-litre jug you can measure exactly 1 litre: fill the 7-litre jug five times and pour out the 17-litre jug twice ($5 \cdot 7 - 2 \cdot 17 = 1$). Here the “jugs” are the exponents. Ciphertext c_1 is m raised to the power 17, and c_2 is m raised to the power 7. Multiplying five copies of c_2 and “removing” two copies of c_1 leaves m raised to the power $5 \cdot 7 - 2 \cdot 17 = 1$, that is m itself.

When this arises. Generating a fresh modulus $n = pq$ for every user is slow, because finding large primes dominates the cost. A careless authority might generate n once and hand each user a different exponent pair (e_i, d_i) over that same n , thinking the different exponents keep users separate. This was a real anti-pattern in some early multi-user and embedded key-management designs.

Setup. One shared modulus n ; two users with $\gcd(e_1, e_2) = 1$; the same message sent to both:

$$c_1 = m^{e_1} \bmod n, \quad c_2 = m^{e_2} \bmod n.$$

Theorem 2. If $\gcd(e_1, e_2) = 1$, anyone who knows (n, e_1, e_2, c_1, c_2) recovers m with no private key.

Why it works.

1. Extended Euclid on (e_1, e_2) gives integers a, b with $ae_1 + be_2 = 1$.
2. Then $c_1^a c_2^b \equiv m^{ae_1} m^{be_2} = m^{ae_1 + be_2} = m \pmod{n}$, by exponent laws alone.
3. One of a, b is negative. For a negative exponent, first compute the modular inverse of the corresponding ciphertext (again by Extended Euclid), then raise to the absolute value.

Notice that the attacker never factors n and never touches d_1 or d_2 .

Worked example. The classic toy key $n = 3233 = 61 \cdot 53$, with $e_1 = 17$, $e_2 = 7$ and secret $m = 42$:

$$c_1 = 42^{17} \bmod 3233 = 2557, \quad c_2 = 42^7 \bmod 3233 = 240.$$

Step 1 — Euclid on $(17, 7)$. $17 = 2 \cdot 7 + 3$, $7 = 2 \cdot 3 + 1$. Back-substituting: $1 = 7 - 2 \cdot 3 = 7 - 2(17 - 2 \cdot 7) = 5 \cdot 7 - 2 \cdot 17$. So $a = -2$, $b = 5$.

Step 2 — handle the negative exponent. $c_1^{-1} \bmod 3233 = 3123$ (check: $2557 \cdot 3123 \equiv 1$).

Step 3 — combine.

$$m \equiv (c_1^{-1})^2 \cdot c_2^5 \equiv 2401 \cdot 1904 \equiv 42 \pmod{3233}.$$

An even worse consequence. Any *insider* who legitimately owns one pair (e_1, d_1) on the shared n can use the fact that $e_1 d_1 - 1$ is a multiple of $\varphi(n)$ to factor n . After that, every other user’s private key is compromised too, not just one message.

Takeaway and fix

Never share a modulus across users or purposes, even with different exponents. It is broken as soon as two exponents on it are coprime, which is the typical case.

3. Bleichenbacher’s Padding Oracle Attack

Picture it

Picture the hidden number m as a runner on a circular track of length n . A bouncer stands at one narrow gate (the window $[2B, 3B)$) and answers only “yes, someone is in the gate” or “no”. You cannot see m . But you can make the runner go s times faster than m does — that is just multiplying by s — and ask whether the sped-up runner is now in the gate. A “yes” means the true m lies in one of a few thin, evenly spaced slices of the track. Choose the next s so that only one slice remains consistent, repeat, and the slices shrink like a game of Twenty Questions until only m is left.

When this arises. A server decrypts RSA ciphertexts that were padded before encryption, and — by an error message, a dropped connection, or even a timing difference — lets the sender tell whether the padding was valid. Nobody intends to leak anything else, but it is very easy to do by accident: TLS servers using the PKCS#1 v1.5 padding standard commonly sent a distinguishable “bad padding” alert. This is not history. Refinements (Bardou et al., 2012; the ROBOT attack, 2017) found major production web servers still exploitable nearly twenty years after the original 1998 paper, and it remains one of the most consequential vulnerabilities ever found in a widely deployed protocol.

What the oracle checks. PKCS#1 v1.5 formats a k -byte block as

$$EB = 0x00 \parallel 0x02 \parallel PS \parallel 0x00 \parallel D,$$

where D is the message and PS is nonzero random padding filling the block to k bytes. For example, with $k = 8$, $D = 0x1234$ and $PS = 0xA1\ 7F\ 3C$:

$$EB = 00\ 02\ A1\ 7F\ 3C\ 00\ 12\ 34.$$

A **padding oracle** answers one yes/no question about a ciphertext c' of the attacker’s choosing: *does $(c')^d \bmod n$ start with 00 02?*

Read as a big integer, “starts with 00 02” simply means the value lies in $[2B, 3B)$, where $B = 256^{k-2}$. That is a very narrow window: a random ciphertext passes only about 1 time in 65,536. The attack is a way of asking this same narrow question about *cleverly chosen* values instead of random ones.

The two ingredients.

- *Malleability.* From $c = m^e \bmod n$ anyone can compute $c \cdot s^e \bmod n$ for any chosen s . This is a valid encryption of $m \cdot s \bmod n$, and it needs no secret.
- *A YES is a strong clue.* If the oracle says YES for $c \cdot s^e$, then $ms \bmod n \in [2B, 3B)$ for an s the attacker knows.

From one YES to a small set of intervals for m . Unwrapping “mod n ”, a YES for s means $ms = rn + t$ with $t \in [2B, 3B)$ for some unknown whole number r (how many laps the fast runner made). Dividing by s :

$$m \in \left[\frac{2B + rn}{s}, \frac{3B - 1 + rn}{s} \right] \quad \text{for some integer } r.$$

Early on several values of r are all consistent, so one YES can leave more than one candidate interval (this really happens, as the example shows). Later YES/NO answers eliminate the wrong ones.

The algorithm in three moves.

1. **Get started.** If c is not itself oracle-valid, find a small s_0 with $c \cdot s_0^e$ oracle-valid and work with ms_0 instead. (If c is already validly padded, $s_0 = 1$.)

2. **Find the next s_i .** While several intervals survive, try s upward one at a time until the oracle says YES. Once only one interval $[a, b]$ survives, jump straight to s values computed from its endpoints; this roughly doubles s each round and halves the interval.
3. **Shrink.** For each surviving interval and each possible lap count r , compute the new sub-interval from the formula above, keep the non-empty ones, and merge overlaps. Stop when one interval containing a single integer remains. That integer is EB (undo s_0 if it was used).

Worked example (computed and verified). Toy parameters: $n = 16,826,323$ ($p = 4093$, $q = 4111$), $e = 17$, $d = 10,882,313$, block length $k = 4$, so $B = 256^2 = 65,536$ and the valid window is $[131,072, 196,608]$. The secret is $m = 150,000$ (hex `0x0249F0`, which starts `00 02`, so $s_0 = 1$), with ciphertext $c = 8,054,451$.

The first query in detail. The attacker tries $s = 1, 2, 3, \dots$ and the first YES comes at $s_1 = 450$. Why? Because

$$450 \cdot 150,000 = 67,500,000 = 4 \cdot n + 194,708,$$

and 194,708 lies in the window. The attacker does not know this, but sees the YES and reasons: “ $450m$ made r laps and landed in the window.” Two lap counts fit the information so far, $r = 4$ and $r = 5$, giving the two intervals in row 1. The true one is $r = 4$.

i	s_i	surviving interval(s) for m	width
0	—	$[131,072, 196,607]$	(initial)
1	450	$[149,859, 150,004]$ and $[187,251, 187,396]$	two intervals
2	562	$[149,934, 150,004]$	70
3	1,235	$[149,977, 150,004]$	27
4	2,581	$[149,995, 150,004]$	9
5	5,610	$[149,995, 150,002]$	7
6	11,331	$[149,995, 150,000]$	5
7	22,885	$[149,998, 150,000]$	2
8	45,881	$[149,999, 150,000]$	1
9	91,873	$[150,000, 150,000]$	0 — recovered

Two things to point out to students. First, the false interval from $r = 5$ is killed at $i = 2$: the next YES ($s = 562$) is consistent only with the true interval. Second, once one interval remains, s_i roughly doubles each round ($562 \rightarrow 1235 \rightarrow 2581 \rightarrow 5610 \rightarrow \dots$) and the interval width roughly halves: this is binary search. Nine oracle queries against a modulus above sixteen million pinned m down exactly.

A caveat about the toy numbers. Here n is a 24-bit number, so the block’s leading `00` byte is automatic and the oracle really checks only the second byte (about 1 random value in 256 passes). With a real 2048-bit modulus both bytes must match (about 1 in 65,000), so a real attack needs thousands to about a million queries rather than nine. The mechanics are identical.

Takeaway and fix

Never let the reason for a decryption failure be distinguishable — by error content, timing, or connection behavior. The complete fix is to stop using PKCS#1 v1.5 encryption in new designs and use OAEP (Lesson 3), whose security proof rules out this whole class of attack.

4. Shared-Prime (Batch GCD) Attack

Picture it

Factoring a single 2048-bit number is like trying to un-bake a cake: hopeless. But if two cakes were baked with a common ingredient, you can find it just by comparing them — and Euclid’s algorithm does that comparison in a fraction of a millisecond. If two RSA moduli happen to share a prime, computing $\gcd(n_1, n_2)$ hands you that prime, and dividing gives the other prime of each. Both keys are broken, and neither owner did anything obviously wrong.

When this arises. Devices such as routers, firewalls and other embedded systems often generate their RSA key the first time they boot. At that moment the operating system’s random number generator has had almost no entropy (no disk, no keyboard, no network jitter yet), so many devices produce *the same* first prime p . A little entropy trickles in (a clock tick, a packet) before the second prime is chosen, so their q ’s differ. The result is many public keys of the form $n_i = p q_i$ with a common p .

This is not hypothetical. In 2012 two independent teams (Lenstra et al., “*Ron was wrong, Whit is right*”; Heninger et al., “*Mining your Ps and Qs*”) collected millions of public RSA keys from the internet and factored thousands of them this way, with no ciphertext and no private key needed — just public moduli. To avoid comparing every pair, they used “batch GCD”, a product-tree algorithm that checks millions of moduli against each other in hours.

Setup. The attacker has two public moduli

$$n_1 = p q_1, \quad n_2 = p q_2, \quad q_1 \neq q_2,$$

and knows nothing else about the factorizations.

Theorem 3. *Then $\gcd(n_1, n_2) = p$. Consequently $q_1 = n_1/p$ and $q_2 = n_2/p$, and the private keys of both moduli can be computed.*

Why it works.

1. p divides both n_1 and n_2 , so $p \mid \gcd(n_1, n_2)$.
2. Since q_1 and q_2 are distinct primes (and differ from p), $\gcd(n_1, n_2) = p \cdot \gcd(q_1, q_2) = p$.
3. With all primes known, compute $\varphi(n_1) = (p-1)(q_1-1)$ and $d_1 = e_1^{-1} \bmod \varphi(n_1)$ — exactly the recipe the legitimate key owner used. The same works for n_2 .

The gcd takes $O(\log^2 n)$ bit operations, whereas factoring a 2048-bit modulus directly is infeasible. That gap is the entire attack.

Worked example. Alice’s key is $n_1 = 3233 = 61 \cdot 53$ and Bob’s key is $n_2 = 3599 = 61 \cdot 59$; both use $e = 17$. Eve sees only the two public moduli, so all she starts with is 3233 and 3599.

Step 1 — Euclid.

$$\begin{aligned} 3599 &= 1 \cdot 3233 + 366, \\ 3233 &= 8 \cdot 366 + 305, \\ 366 &= 1 \cdot 305 + 61, \\ 305 &= 5 \cdot 61 + 0, \end{aligned}$$

so $\gcd(3233, 3599) = 61$. Eve has found the shared prime $p = 61$ in four division steps.

Step 2 — factor both keys. $q_1 = 3233/61 = 53$ and $q_2 = 3599/61 = 59$.

Step 3 — rebuild Alice’s private key. $\varphi(n_1) = 60 \cdot 52 = 3120$ and $d_1 = 17^{-1} \bmod 3120 = 2753$ (check: $17 \cdot 2753 = 46801 = 15 \cdot 3120 + 1$). Bob’s key falls the same way: $\varphi(n_2) = 60 \cdot 58 = 3480$, $d_2 = 1433$.

Step 4 — read a message. Suppose someone sent Alice the ciphertext $c = 2183$. Eve computes $2183^{2753} \bmod 3233 = 1234$, the plaintext, without ever being given a key.

Takeaway and fix

Generate keys only after the system has gathered enough entropy, use the operating system’s cryptographic random number generator, and draw p and q from fresh randomness. Avoid generating keys on the first boot of a device with no entropy source. Operators can also check their own newly generated keys against large public key collections for shared factors. Notice the lesson: RSA’s security depends not only on the math but on the quality of the random numbers used to pick the primes.

5. Fermat’s Factorization (Close Primes)

Picture it

Think of $n = pq$ as the area of a $p \times q$ rectangle of unit tiles. Any rectangle can be rearranged into a big square with a smaller square cut out of one corner. Let a be the *midpoint* of p and q and b the *distance from the midpoint to each*, so $p = a - b$ and $q = a + b$. Then the rectangle has the same area as a square of side a minus a square of side b :

$$pq = (a - b)(a + b) = a^2 - b^2.$$

If p and q are nearly equal, the rectangle is almost a square already, so a is only slightly above $\sqrt{n}$ and the cut-out square is tiny. The attacker simply tries $a = \lceil \sqrt{n} \rceil, \lceil \sqrt{n} \rceil + 1, \dots$ until $a^2 - n$ turns out to be a perfect square. That is a search from a known starting point, not a blind one.

When this arises. A key generator picks one random prime p and then, to save time or because of a flawed random source, picks q close to it — for instance the next prime after p , or p plus a small random offset. The idea itself dates to 1643, and for a long time it was regarded as a textbook curiosity. In 2022 Hanno Böck applied it to large collections of real public keys and found vulnerable ones: some Canon and Fujifilm (Fuji Xerox) printers generated keys with a cryptographic module that did not randomize its two primes enough (the Canon issue is tracked as CVE-2022-26351). It is a rare flaw, but it is real, and it needs nothing except the public modulus.

Setup. The attacker knows only $n = pq$ with $p < q$ odd primes. Nothing else.

Theorem 4. Let $a = \frac{p+q}{2}$ and $b = \frac{q-p}{2}$ (integers, since p, q are odd). Then $a^2 - n = b^2$, and a is the smallest integer $\geq \sqrt{n}$ for which $a^2 - n$ is a perfect square. Testing $a = \lceil \sqrt{n} \rceil, \lceil \sqrt{n} \rceil + 1, \dots$ therefore finds a after at most

$$\frac{(q-p)^2}{8\sqrt{n}} + 1$$

tries, and then $p = a - b$, $q = a + b$.

Why it works.

1. Algebra: $a^2 - b^2 = (a - b)(a + b) = p \cdot q = n$, so $a^2 - n = b^2$ is a perfect square.

- Conversely, if $a^2 - n = b^2$ for any integers $a > b \geq 0$, then $n = (a - b)(a + b)$ is a factorization. Since $n = pq$ has only one pair of nontrivial factors, the first hit gives exactly p and q .
- How many tries? The search starts at $\lceil \sqrt{n} \rceil \geq \sqrt{n}$ and stops at $a = \frac{p+q}{2}$, so it takes at most $\frac{p+q}{2} - \sqrt{pq} + 1$ steps. Now

$$\frac{p+q}{2} - \sqrt{pq} = \frac{(\sqrt{q} - \sqrt{p})^2}{2} = \frac{(q-p)^2}{2(\sqrt{q} + \sqrt{p})^2} \leq \frac{(q-p)^2}{8\sqrt{n}},$$

because $(\sqrt{p} + \sqrt{q})^2 = p + q + 2\sqrt{n} \geq 4\sqrt{n}$ (as $p + q \geq 2\sqrt{pq}$).

So the work depends on the gap $q - p$, and grows with its square.

Worked example. Eve sees the public key $n = 5959$, $e = 17$. Here $\sqrt{5959} \approx 77.2$, so she starts at $a = 78$:

a	a^2	$a^2 - 5959$	perfect square?
78	6084	125	no ($11^2 = 121$, $12^2 = 144$)
79	6241	282	no ($16^2 = 256$, $17^2 = 289$)
80	6400	441	yes , $441 = 21^2$

So $a = 80$, $b = 21$, and $p = 80 - 21 = 59$, $q = 80 + 21 = 101$. Check: $59 \cdot 101 = 5959$. Three tries, and no factoring in the usual sense.

Finishing the break. $\varphi(n) = 58 \cdot 100 = 5800$ and $d = 17^{-1} \bmod 5800 = 3753$ (check: $17 \cdot 3753 = 63801 = 11 \cdot 5800 + 1$). If Alice's ciphertext is $c = 2029$, Eve computes $2029^{3753} \bmod 5959 = 1234$, the plaintext.

Contrast: primes that are far apart. Take $p = 101$, $q = 1009$, so $n = 101909$ and $\sqrt{n} \approx 319.2$. Now a must climb all the way to $\frac{101+1009}{2} = 555$ (with $b = 454$), which is 236 tries, against a bound of $\frac{908^2}{8\sqrt{n}} + 1 \approx 323$. The bigger the gap, the longer the climb.

How big a gap is safe? For a 2048-bit modulus, $8\sqrt{n} \approx 2^{1027}$. The bound above then says:

Gap $q - p$	Tries needed (at most)
below about 2^{512}	1
2^{512+t}	about $2^{2t}/8$ (each extra bit of gap quadruples the work)
about 2^{1023} (two independent random 1024-bit primes)	about 2^{1019} , hopeless

So Fermat only works when the two primes agree in roughly their top half of bits. Independent random primes never do.

Try it (about 10 lines of Python).

```
from math import isqrt

def fermat(n):
    a = isqrt(n)
    if a * a < n:          # a = ceil(sqrt(n))
```

```

    a += 1
while True:
    b2 = a * a - n
    b = isqrt(b2)
    if b * b == b2:      # a^2 - n is a perfect square
        return a - b, a + b
    a += 1

```

`fermat(5959)` returns (59, 101). A good classroom demo: generate two 512-bit primes with `p = nextprime(r)` and `q = nextprime(p + 10**9)`. The 1024-bit modulus falls on the very first try.

Takeaway and fix

Choose p and q *independently* from a good random source, and reject the pair if they are too close. Standards such as FIPS 186-4 require $|p - q| > 2^{m_{\text{len}}/2 - 100}$. With independent random primes this is satisfied automatically, so the attack only ever hits a faulty generator, never a correct one.

6. Pollard's $p - 1$ Factoring Method

Picture it

Look at the powers of a number a on a clock with p hours. By Fermat's little theorem the sequence $a, a^2, a^3, \dots$ lands back on 1 exactly when the exponent is a multiple of (the period dividing) $p - 1$. Now suppose $p - 1$ is built only from small steps, such as $2^4 \cdot 3^2 \cdot 7$. Then a number like $B! = 1 \cdot 2 \cdot 3 \cdots B$ is a multiple of $p - 1$, so $a^{B!}$ reads 1 on the p -clock. The attacker cannot see the p -clock — all she has is $n = pq$ — but she can compute $a^{B!} \bmod n$ anyway. The other prime q has its own, much longer period $q - 1$, so on the q -clock the same number almost surely does *not* read 1. Subtracting 1 and taking a gcd with n cleanly separates the two clocks and reveals p .

Vocabulary. A number is B -smooth if all its prime factors are at most B . For example $1008 = 2^4 \cdot 3^2 \cdot 7$ is 7-smooth, while $1018 = 2 \cdot 509$ is not even 100-smooth.

When this arises. A key generator builds a prime from small pieces, say $p = 2m + 1$ where m is a product of small primes, because such a form makes it easy to prove p prime. The resulting $p - 1$ is smooth. Standards from the 1990s therefore asked for “strong primes”, and modern ones such as FIPS 186 still require $p - 1$ to contain a large prime factor. John Pollard published the method in 1974, and it remains one of the first things factoring software tries on a number, because it costs almost nothing.

Setup. The attacker knows only $n = pq$ and picks a bound B (a guess about how smooth $p - 1$ might be).

Theorem 5. Let $M = B!$ and a coprime to n . Suppose $p - 1$ divides M but $a^M \not\equiv 1 \pmod{q}$. Then

$$\gcd(a^M - 1, n) = p.$$

Why it works.

1. Write $M = (p - 1)t$. By Fermat's little theorem $a^{p-1} \equiv 1 \pmod{p}$, so $a^M = (a^{p-1})^t \equiv 1 \pmod{p}$. Hence $p \mid a^M - 1$, and $p \mid n$, so $p \mid \gcd(a^M - 1, n)$.

2. By assumption $a^M \not\equiv 1 \pmod{q}$, so $q \nmid a^M - 1$. The gcd is therefore a divisor of $n = pq$ that contains p but not q , so it equals p .
3. The attacker never needs p to compute this: $a^M \bmod n$ is an ordinary modular exponentiation, and $\gcd((a^M \bmod n) - 1, n)$ equals $\gcd(a^M - 1, n)$.

The algorithm. Start with $a = 2$. For $j = 2, 3, 4, \dots$ replace a by $a^j \bmod n$. After step j we have $a \equiv 2^{j!} \pmod{n}$, because $(2^{(j-1)!})^j = 2^{j!}$. After each step compute $d = \gcd(a - 1, n)$:

- $d = 1$: not yet; keep going (up to B).
- $1 < d < n$: success, d is a prime factor.
- $d = n$: both $p - 1$ and $q - 1$ divide $j!$ at the same time. Retry with a different base a .

Worked example 1 (by hand). $n = 299$. In fact $299 = 13 \cdot 23$, with $13 - 1 = 12 = 2^2 \cdot 3$ (smooth) and $23 - 1 = 22 = 2 \cdot 11$. Eve does not know this.

j	$a = 2^{j!} \bmod 299$	$a - 1$	$\gcd(a - 1, 299)$
2	$2^2 = 4$	3	1
3	$4^3 = 64$	63	1
4	$64^4 \equiv 27$	26	13

(For $j = 4$: $64^2 = 4096 \equiv 209$, and $209^2 = 43681 \equiv 27 \pmod{299}$.) At $j = 4$ we have $j! = 24$, and $12 \mid 24$. So $2^{24} \equiv 1 \pmod{13}$; indeed $27 = 2 \cdot 13 + 1$. But $22 \nmid 24$, and indeed $27 \equiv 4 \pmod{23}$, not 1. The gcd isolates 13, and $299/13 = 23$.

Worked example 2 (a small RSA key). $n = 1028171 = 1009 \cdot 1019$, $e = 17$. The hidden structure is $1009 - 1 = 1008 = 2^4 \cdot 3^2 \cdot 7$ and $1019 - 1 = 1018 = 2 \cdot 509$. Since $1008 \mid 7! = 5040$ (indeed $5040 = 5 \cdot 1008$), the attack should succeed at $j = 7$, but 509 is a prime factor of $q - 1$ that $7!$ cannot cancel.

j	$a = 2^{j!} \bmod n$	$\gcd(a - 1, n)$
2	4	1
3	64	1
4	326 480	1
5	321 438	1
6	709 177	1 (720 is not a multiple of 1008)
7	615 491	1009 (5040 = 5 · 1008)

So $p = 1009$, $q = n/p = 1019$, and with $\varphi(n) = 1008 \cdot 1018$ Eve computes $d = e^{-1} \bmod \varphi(n)$ and owns the key. Seven modular exponentiations did what trial division by about 1000 numbers would have done here, but the point is how it scales.

Scaling up. The work is only about B small modular exponentiations, no matter how large p is. In a test with a 754-bit modulus, whose smaller prime p (242 bits) had $p - 1$ made of primes below 300 and whose other prime was a random 512-bit prime, the method returned p at $j = 281$ in a fraction of a second. By contrast, on a modulus with a 512-bit *safe* prime ($p = 2p' + 1$ with p' prime), the same code with $B = 3000$ found nothing.

Try it (Python).

```
from math import gcd
```

```
def pollard_p_minus_1(n, B=100000):
    a = 2
    for j in range(2, B + 1):
        a = pow(a, j, n)          # now a = 2^(j!) mod n
        d = gcd(a - 1, n)
        if 1 < d < n:
            return d              # a nontrivial factor
        if d == n:
            return None           # both p-1, q-1 smooth: change base
    return None                  # B too small
```

`pollard_p_minus_1(299)` returns 13, and `pollard_p_minus_1(1028171)` returns 1009.

What stops it. If $p - 1$ has a large prime factor r , the attacker needs $B \geq r$ before $B!$ becomes a multiple of $p - 1$. For a safe prime $p = 2p' + 1$ this means $B \geq p' \approx p/2$, which costs as much as trial division and is hopeless. A random 1024-bit prime has $p - 1$ smooth up to $B = 2^{20}$ with a probability far below 2^{-100} , so smoothness is essentially never an accident. The danger comes only from generators that build p so that $p - 1$ is smooth, on purpose or by structure.

Takeaway and fix

Generate primes so that $p - 1$ (and $q - 1$) contains a large prime factor, as FIPS 186 key generation does. Safe primes $p = 2p' + 1$ achieve this but are not required; random large primes already work with overwhelming probability. Together with Section 5 the lesson is the same: the key generator must produce p and q that are *large, independent, and unstructured*, because RSA's security rests on n looking like a random product of two large primes, not merely on n being large.

Where each tool came from. All six attacks reuse Lesson 1 material as *attack* tools rather than construction tools: CRT (attack 1), the Extended Euclidean Algorithm (attack 2), multiplicative malleability (attack 3), the plain Euclidean gcd (attack 4), the difference of two squares (attack 5), and Fermat's little theorem together with the gcd (attack 6). Attacks 1 and 2 exploit a *misuse* of correctly generated keys; attack 3 exploits an *information leak*; attacks 4, 5 and 6 exploit *flawed key generation*. Attacks 5 and 6 are the only ones that actually factor n , and they succeed only when p and q are not independent, unstructured, random primes.